

Drivo Scrum Update Script

This document provides a structured guide and speaking points for your upcoming Scrum standup. It highlights what has been built, how the architecture is set up, and what is planned next.

Standup Script (Elevator Pitch Style)

*"Hi everyone. Here is the update on **Drivo** (our premium sustainable EV ride-sharing platform). We've successfully laid down the core infrastructure and design systems in Week 1, paving the way for the core booking features next.*

Here is what is completed:

1. **Design System Integration:** Sourced design tokens from our `DESIGN.md` spec directly into Tailwind CSS v4, setting up our premium green/deep blue brand colors, the 'Comfort Radius' shapes (16px/24px), and the baseline layout.
2. **Supabase Database Schema:** Defined schemas for `riders`, `drivers`, `vehicles`, and `admins`. Implemented strict Row-Level Security (RLS) policies so users can only access their own profile records, while allowing riders to query driver/vehicle details during active rides.
3. **Role-Aware Authentication Flow:** Integrated Supabase OTP (One-Time Password) email authentication. Built a custom `AuthProvider` that checks all three user roles in parallel and routes them dynamically.
4. **Smart Routing System:** Created a client-side routing model in React Router v7. It features a central `/` redirection router that checks role status and sends users to the Rider Home, Driver Home, Admin Dashboard, or registration/profile completion pages automatically.
5. **Onboarding & Auth UI:** Fully completed the interactive Login (OTP verification) and Profile Setup (registration) pages with client-side validation and styling.

Next up, we will be integrating the map interface, tracking drivers in real time, and setting up the active ride booking sheets."

Screen-by-Screen Talking Points

Here is a list of things to discuss for each screen when showing them or explaining progress:

1. Login Screen (`LoginPage.jsx`)

- What to say:

"For the login experience, we've implemented a passwordless authentication flow using Supabase email OTP. This minimizes friction for our users.

The screen has two states: Email Input and OTP Code Verification. It's built with strict visual guidelines from our design spec, including a 20px side margin for optimal thumb spacing on mobile, custom ghost-style inputs, and a custom numeric keyboard setup for entering the 6-digit code. If a user is already logged in, the route guards will automatically bypass this screen."

2. Complete Profile / Registration Screen (`RegisterPage.jsx`)

- What to say:

"Once a user authenticates for the first time, they are redirected to this screen to complete their registration.

This screen collects the user's Full Name and Mobile Number. We've locked the country code selector to `+91` with a formatted input to prevent data inconsistency. When the form is submitted, it inserts a new rider record into the Supabase database. Once written, our auth state shifts to reflect a fully registered 'rider' and sends them to the home screen."

3. Rider Home Page (`rider/HomePage.jsx`)

- What to say:

"This is the landing view for authenticated riders. The route is protected by our `RoLeRoute` component, ensuring only users with rider profiles can access it.

Right now, this acts as our authenticated entry point dashboard. It displays the active rider's email and a sign-out mechanism. We are fully ready to drop in our Google Maps API, EV search widgets, and ride selection cards here in the next sprint."

4. Driver Dashboard (`driver/HomePage.jsx`)

- What to say:

"Similar to the Rider view, this is the dashboard entry point for drivers. It is locked behind driver role security.

Currently, it shows the driver's email and acts as a shell. In the next phase, we'll hook this up to driver status toggles ('Online' / 'Offline') and a real-time ride request queue where drivers can accept or decline trips."

5. Admin Dashboard (`admin/DashboardPage.jsx`)

- What to say:

"This is our control center dashboard for Drivo administrators, accessible only to accounts configured in our `admins` table.

It will host our driver registration approval queue and general ride reports. Next week, we'll build out the approval interface so admins can review pending driver licenses and toggle their status to 'active' or 'suspended'."

Technical Details (If anyone asks questions)

- **Database (Supabase):**
- Check the schema at `schema.sql`. We have RLS policies set up to secure rider and driver tables.
- Rider profiles write to `public.riders`, driver profiles to `public.drivers`, and admin records to `public.admins`.
- **State Management:**
- Check our React custom hook `useAuth.jsx` which checks all three tables (`riders`, `drivers`, `admins`) in parallel on login to identify user roles and holds the loading state.
- **Routing & Guards:**
- Check `App.jsx`. It uses `SmartRedirect` as a controller at `/`. Public routes bounce logged-in users, unregistered routes allow only profile completion, and role-specific routes guard dashboards.
- **Styling:**
- Check `index.css`. It leverages Tailwind CSS v4 variables mapping the Geist font, primary dark/forest green theme colors, and the 'Comfort Radius' component styling.